

Efficientamento e Deployment degli LLM



by Daniele Mantovani



LLM Distillation

Comprimere la conoscenza di un modello grande in uno piccolo ed efficiente

L'IDEA

Teacher
es. 70B param.

insegna a

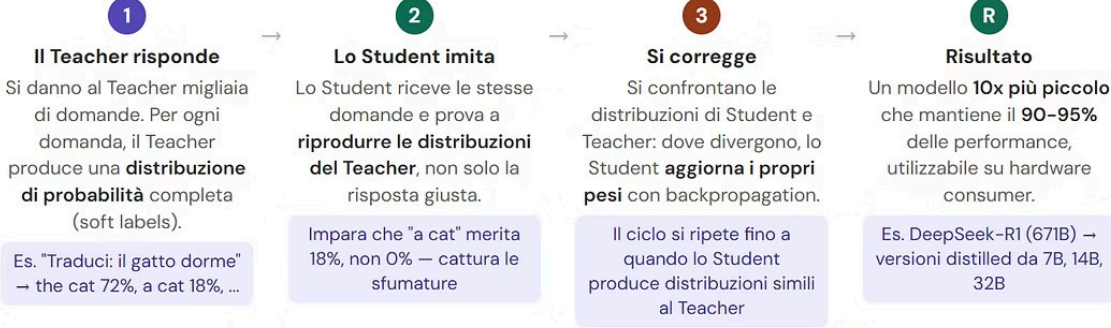
Student
es. 7B param.

Invece di servire il modello grande e costoso, si addestra un **modello piccolo che ne imita il comportamento**.

Il segreto: lo Student non impara solo le risposte giuste, ma **come ragiona il Teacher** — attraverso le sue distribuzioni di probabilità.

Analogia: un professore esperto non dice solo "la risposta è 42" — spiega il ragionamento. Le soft labels sono quel ragionamento.

COME FUNZIONA: IL WORKFLOW DI DISTILLAZIONE



SOFT LABELS VS HARD LABELS — IL CUORE DELLA DISTILLAZIONE

Hard Label (etichetta classica)

"Traduci: il gatto dorme"

the cat **100%**
a cat **0%**
the kitten **0%**
le chat **0%**

Dice solo **quale risposta è giusta**. Tutto il resto vale zero: nessuna sfumatura.

Soft Label (dal Teacher)

"Traduci: il gatto dorme"

the cat **72%**
a cat **18%**
the kitten **10%**
le chat **0%**

Rivela **le relazioni tra le alternative**: "a cat" è quasi giusto, "the kitten" ci si avvicina, "le chat" è sbagliato.

VARIANTI NEL CONTESTO LLM

Off-Policy (standard)

Student si addestra su output pre-generati dal Teacher. Semplice ed economico.

On-Policy

Student genera i propri token, il Teacher dà feedback. Più preciso ma molto più costoso.

Rappresentazioni

Student replica anche le attivazioni intermedie dei layer, non solo l'output finale.

VANTAGGI

- + Inferenza efficiente: meno parametri, memoria, latenza
- + Mantiene ~90-95% delle performance del Teacher
- + Democratizza l'accesso ai modelli potenti

LIMITI

- Capacity gap: lo Student non cattura tutta la conoscenza
- Costa comunque far "parlare" il Teacher su tutto il dataset
- La qualità dello Student dipende interamente dal Teacher

Quantizzazione di un LLM

Ridurre la precisione dei pesi per risparmiare memoria, latenza e costi

COS'È LA QUANTIZZAZIONE

16/32 bit

 →

4/8 bit

16.321, -0.033... → 1, -3, 4, 1...

Ogni peso del modello è un numero decimale memorizzato con **16 o 32 bit di precisione**. La quantizzazione li rappresenta con **meno bit** (8, 4, o anche 2), accettando una piccola perdita di precisione.

↓ Memoria

↓ Latenza

↓ Costo

DUE PARADIGMI

PTQ
Post-Training Quantization

Si prende un modello **già addestrato** e si riducono i bit dei pesi senza ri-addestrarlo. Pratico e veloce: basta un piccolo dataset di calibrazione.

Approccio dominante per LLM

Tecniche chiave: GPTQ, AWQ, GGUF

QAT
Quantization-Aware Training

Si **simula la quantizzazione durante il training**, permettendo al modello di adattarsi. Risultati migliori, ma costo computazionale molto più alto.

Qualità superiore, molto più costoso

Usato quando la qualità è critica

TRADE-OFF PER UN MODELLO 7B (ES. LLAMA 3 8B)

PRECISIONE	MEMORIA	QUALITÀ VS BASELINE	COMPRESSIONE
BF16	~14 GB	100% (baseline)	1x
INT8	~7 GB	~99%	2x
INT4	~3.5 GB	~95%	4x
INT2	~1.75 GB	Degrado significativo	8x

LO STANDARD ATTUALE: 4 BIT CON QUANTIZZAZIONE A GRUPPI

3 1 4 2

2 5 1 3

gruppo 1

7 2 6 1

4 3 5 8

gruppo 2

Invece di usare un unico fattore di scala per tutti i pesi, si dividono i pesi in **piccoli gruppi** (es. 32-128 pesi) e ogni gruppo ha il proprio fattore di scala.

Risultato: **compressione 4x** con perdite di qualità generalmente accettabili. È lo standard de facto per l'esecuzione locale di LLM (formato GGUF, Q4_K_M).

DISTILLAZIONE VS QUANTIZZAZIONE: TECNICHE COMPLEMENTARI

Distillazione (slide precedente): crea un **modello diverso**, più piccolo, addestrato a imitare quello grande. Cambia l'architettura.

Quantizzazione (questa slide): prende lo **stesso modello** e ne riduce la precisione numerica dei pesi. Non cambia l'architettura.

Distillazione + Quantizzazione

Spesso usate insieme

ESEMPIO CONCRETO



Punto chiave: un modello 7B passa da 14 GB (BF16) a 3.5 GB (INT4) — entra in una GPU consumer con ~95% della qualità originale

Il Formato GGUF

Lo standard de facto per eseguire LLM in locale con llama.cpp

COS'È GGUF

GGUF (GGML Universal Format) è un **formato file singolo** che impacchetta tutto il necessario per eseguire un LLM: architettura, parametri di quantizzazione e pesi del modello.

A differenza di SafeTensors (che contiene solo i pesi), GGUF è **autocontenuto**: un solo file = un modello completo pronto all'uso.

Creato da Georgi Gerganov, evoluzione del formato GGML. Progettato per llama.cpp.

STRUTTURA DEL FILE

HEADER	Magic number ("GGUF") + numero di versione Identifica il formato e garantisce compatibilità
METADATA	Key-value store flessibile: architettura, context length, dimensione embedding, numero layer, tokenizer Contiene tutto ciò che serve per ricostruire il modello vantaggio vs SafeTensors
TENSORI	Pesi del modello — caricati tramite memory-mapping (mmap) Non vengono copiati in RAM: il sistema operativo li mappa direttamente dal disco, rendendo il caricamento molto veloce

HARDWARE SUPPORTATO — DAL LAPTOP AL SERVER

x86 (CPU) ARM (Mac, mobile) CUDA (NVIDIA)
Metal (Apple) Vulkan OpenCL HIP (AMD) SYCL (Intel)

TIPI DI QUANTIZZAZIONE DISPONIBILI

F32 / F16 Floating point, nessuna quantizzazione. Massima qualità.	Q8_0 8 bit base. Qualità quasi identica, memoria dimezzata.	Q4_0 4 bit base. Buon compromesso, compressione 4x.
Q4_K_M * 4 bit "K-quant". Blocchi da 256 con scala FP16. Il migliore rapporto qualità/dimensione.	Q5_K / Q6_K 5-6 bit K-quant. Qualità superiore a Q4, file leggermente più grandi.	IQ2_XS ~2 bit, sperimentale. Compressione estrema, degrado importante.

* **Q4_K_M** è la scelta raccomandata per la maggior parte degli usi — miglior rapporto qualità/dimensione

ECOSISTEMA E WORKFLOW TIPICO

- Partenza:** modello in formato SafeTensors/PyTorch (es. da Hugging Face)
- Conversione:** `convert_hf_to_gguf.py` converte in GGUF F16 (non quantizzato)
- Quantizzazione:** `llama-quantize` comprime al livello desiderato (es. Q4_K_M)
- Esecuzione:** `llama.cpp` / Ollama / LM Studio caricano il file e servono il modello

PERCHÉ GGUF È RILEVANTE

- 1 Portabilità:** un singolo file autocontenuto. Nessun file di configurazione separato, nessuna dipendenza — si copia il file e funziona.
- 2 Efficienza:** memory mapping (mmap) permette di caricare il modello quasi istantaneamente, senza copiarlo tutto in RAM prima. Il sistema operativo gestisce la paginazione.
- 3 Quantizzazione integrata:** il formato supporta nativamente tutti i livelli di quantizzazione. Non serve un passaggio separato — il file è già pronto.
- 4 Hardware universale:** llama.cpp supporta CPU, GPU NVIDIA/AMD/Intel, Apple Silicon. Lo stesso file GGUF gira ovunque.
- 5 Ecosistema maturo:** Ollama, LM Studio, GPT4All — le principali app per LLM locali usano tutte GGUF come formato nativo.

VARIANTI SU HUGGING FACE

Quando scaricate un modello, vedrete nomi come:

`Q4_K_M` `Q4_K_S` `Q5_K_M` `Q8_0`

M = Medium (qualità bilanciata) / **S** = Small (quantizzazione più aggressiva dei layer meno importanti)

MODELLI DISPONIBILI IN GGUF:

Llama Mistral Gemma Qwen Phi DeepSeek + praticamente ogni modello open-weight

Dimensionamento Hardware

Quanta VRAM serve per far girare un LLM? Una formula semplice per rispondere

LA FORMULA

$$\text{VRAM} \approx \frac{P \times B}{8} \times 1.2$$

P

Parametri

Dimensione del modello (es. 8 miliardi)

B

Bit per peso

FP16 = 16, INT8 = 8, INT4 = 4

1.2

Overhead

~20% per KV-Cache e sistema operativo

ESEMPIO PRATICO

Una RTX 4060 (8 GB) può ospitare Llama 3 8B quantizzato a 4 bit?

Dati $P = 8$ miliardi, $B = 4$ bit

Calcolo $\frac{8 \times 10^9 \times 4}{8} \times 1.2 = 4 \times 10^9 \times 1.2 = 4.8 \text{ GB}$

Risultato $4.8 \text{ GB} < 8 \text{ GB (RTX 4060)}$ ✓

4.8 GB di VRAM necessari

Si, entra — restano ~3.2 GB liberi per la KV-Cache di contesti più lunghi

GUIDA PRATICA: COSA GIRA SU QUALE GPU?

GPU	VRAM	TIPO	MODELLO 8B INT4	MODELLO 14B INT4	MODELLO 70B INT4
RTX 4060	8 GB	Consumer	Si (4.8 GB)	Stretto (8.4 GB)	No (42 GB)
RTX 4070 Ti Super	16 GB	Consumer	Si (4.8 GB)	Si (8.4 GB)	No (42 GB)
RTX 4090	24 GB	Consumer	Si (4.8 GB)	Si (8.4 GB)	No (42 GB)
Mac M4 Pro	24 GB*	Unified	Si (4.8 GB)	Si (8.4 GB)	No (42 GB)
Mac M4 Max	64 GB*	Unified	Si (4.8 GB)	Si (8.4 GB)	Si (42 GB)
A100	80 GB	Cloud/Server	Si (4.8 GB)	Si (8.4 GB)	Si (42 GB)
H100	80 GB	Cloud/Server	Si (4.8 GB)	Si (8.4 GB)	Si (42 GB)

* Apple Silicon: memoria unificata condivisa tra CPU e GPU. Disponibile per il modello ma anche per il sistema operativo e le app.

ATTENZIONE: L'OVERHEAD KV-CACHE CRESCE COL CONTESTO

Il fattore **1.2** nella formula assume contesti brevi. Conversazioni lunghe o contesti da **128K+ token** possono richiedere GB aggiuntivi di VRAM solo per la KV-Cache. Per contesti molto lunghi, la VRAM necessaria può essere **significativamente superiore** alla stima della formula.

Regola d'oro: il modello deve stare interamente in VRAM per la massima velocità. Se sfora, l'offloading su CPU funziona ma è molto più lento.

Saturazione della Finestra di Contesto

Cosa succede quando la conversazione diventa troppo lunga per il modello

Il problema: conseguenze dello sfioramento

LA FINESTRA DI CONTESTO SI RIEMPIE

Inizio conversazione (tutto ok)



Dopo molti scambi (si riempie)



Sfioramento (contesto pieno)



TRE CONSEGUENZE

1

Troncamento

I token più vecchi vengono **scartati silenziosamente**. Il modello "dimentica" le istruzioni iniziali e il contesto precedente.

2

Errore OOM

La KV-Cache esaurisce la **VRAM disponibile**. Il processo crasha o rallenta drammaticamente con offloading su CPU.

3

Allucinazioni

Il modello perde coerenza, **dimentica le istruzioni** del system prompt e inventa informazioni per colmare le lacune.

ESEMPIO: CHATBOT AZIENDALE CON CONTESTO DA 4K TOKEN



Le soluzioni: strategie di mitigazione

1. RETRIEVAL-AUGMENTED GENERATION (RAG)

COME FUNZIONA
Domanda →
Cerca →
Frammento →
Risposta

Invece di mettere tutto nel contesto, si forniscono al modello **solo i frammenti rilevanti**, recuperati da una base di conoscenza esterna tramite ricerca semantica. Il contesto resta corto perché contiene solo ciò che serve per rispondere alla domanda corrente.

Soluzione più diffusa in produzione

2. COMPRESSIONE DEL CONTESTO

COME FUNZIONA
Conversazione
lunga →
Riassunto
compatto

Le parti più vecchie della conversazione vengono **riassunte automaticamente** in poche frasi. Il modello lavora con il riassunto invece che con la cronologia completa. Approccio usato da ChatGPT e Claude per conversazioni lunghe: a un certo punto i messaggi iniziali vengono compressi.

Trasparente per l'utente

3. MEMORIA ESTERNA

COME FUNZIONA
Fatti chiave →
Database →
Richiamati
quando serve

I fatti importanti della conversazione vengono estratti e salvati in un **database esterno** (fact store). Ad ogni turno, i fatti rilevanti vengono re-iniettati nel contesto. Simile alla "memoria" di Claude: le informazioni persistono tra conversazioni senza occupare la finestra di contesto.

Memoria persistente tra sessioni

La finestra di contesto è una risorsa finita. Gestirla bene (RAG, compressione, memoria esterna) è una competenza chiave per chi sviluppa applicazioni con LLM.

Confronto Modelli: SLM vs Medium LM vs LLM

Tre categorie per ordine di grandezza: efficienza, equilibrio, potenza

S

Small Language Model

~100M – 3B parametri

ESEMPI

Gemma 3 (2B) Phi-4 Mini Llama 4 (1B/3B) Ministral

CAPACITÀ

- Comprensione linguistica di base
- **Classificazione**, tagging, sentiment
- Prompt semplici e deterministici
- Specializzati con **fine-tuning** mirato

HARDWARE E COSTI

Girano su **CPU**, **smartphone**, **dispositivi edge**. Nessuna GPU necessaria. Latenza bassissima, costi minimi.

Costo: trascurabile

CASI D'USO TIPICI

Classificazione spam / sentiment on-device
Autocompletamento locale
Assistenti embedded (IoT, mobile)
Privacy-first: dati che non lasciano il dispositivo

M

Medium Language Model

~7B – 30B parametri

ESEMPI

Llama 4 (8B/11B) Mistral NeMo 2 GPT-5.2 Mini Haiku 4.5

CAPACITÀ

- **Conversazione coerente** e multi-turno
- Reasoning limitato (catene corte)
- RAG efficace su documenti strutturati
- Buona generalizzazione su **task noti**

HARDWARE E COSTI

GPU singola consumer (quantizzati) o cloud standard. Fine-tuning possibile con **LoRA/QLoRA** su hardware accessibile.

Costo: accessibile

CASI D'USO TIPICI

Chatbot aziendali e assistenti FAQ
Generazione di contenuti (email, report)
Coding assistant per task strutturati
RAG su knowledge base aziendale

L

Large Language Model

~70B – 500B+ parametri

ESEMPI

GPT-5.2 Claude Opus 4.5 Gemini 3.0 Llama 3 70B+

CAPACITÀ

- **Reasoning multi-step** complesso
- Tool use e **agenti autonomi**
- Program synthesis avanzata
- Comprensione **cross-domain** profonda

HARDWARE E COSTI

Cluster di GPU o accesso via API cloud. Training e inferenza richiedono infrastrutture dedicate e costi significativi.

Costo: elevato

CASI D'USO TIPICI

Ricerca e analisi complessa
Agenti autonomi multi-tool
Software engineering avanzato
Task dove la qualità non è negoziabile

La distinzione è relativa e in continua evoluzione

Un modello "grande" nel 2022 (GPT-2, 1.5B) oggi è classificato come "small". La vera scelta non è tra S/M/L, ma tra **il modello più economico che soddisfa i requisiti di qualità** — poi si scala verso l'alto solo dove necessario.



Panorama dei Modelli di Frontiera (Febbraio 2026)

Prezzi API, Benchmark e Strategie di Scelta

Il mercato dei modelli di frontiera nel 2026 è definito da tre dinamiche simultanee: la convergenza delle prestazioni ai vertici (il divario tra primo e decimo posto su MMLU-Pro è inferiore a 7 punti), il crollo dei prezzi (riduzione media dell'80% rispetto al 2024-2025), e l'emergere di alternative open-weight competitive. Non esiste più "il modello migliore" in assoluto: la scelta ottimale dipende dal task, dal budget e dai vincoli di latenza.

SEZIONE 1: TABELLA COMPARATIVA DEI PREZZI API (\$/1M token)

Modello	Provider	Input	Output	Cache Read	Context Window
Claude Opus 4.6	Anthropic	\$15.00	\$75.00	—	200K (1M beta)
Claude Opus 4.5	Anthropic	\$5.00	\$25.00	\$0.50	200K
Claude Sonnet 4.5	Anthropic	\$3.00	\$15.00	\$0.30	200K
Claude Haiku 4.5	Anthropic	\$1.00	\$5.00	\$0.10	200K
GPT-5.2	OpenAI	\$1.75	\$14.00	\$0.175	400K
GPT-5.2 Pro	OpenAI	\$21.00	\$168.00	—	400K
Gemini 2.5 Pro	Google	\$1.25	\$10.00	\$0.125	1M
Gemini 3 Pro (preview)	Google	\$2.00	\$12.00	\$0.20	1M
DeepSeek V3.1	DeepSeek	\$0.15	\$0.75	—	128K
DeepSeek R1	DeepSeek	\$0.55	\$2.19	—	128K
Kimi K2.5	Moonshot AI	\$0.60	\$3.00	\$0.10	262K

Nota: Tutti i prezzi si riferiscono al tier standard (≤200K token per i modelli con pricing a scaglioni). I prezzi possono variare; verificare sempre le pagine ufficiali.

SEZIONE 2: CONFRONTO BENCHMARK

Benchmark	Cosa misura	GPT-5.2	Claude Opus 4.6	Gemini 2.5 Pro	DeepSeek V3.2	Kimi K2.5
SWE-bench Verified	Software Engineering reale	80.0%	80.8%	63.8%	77.8%*	~70%
AIME 2025	Matematica competitiva	100%	100%	92%	89.3%	96.1%
GPQA Diamond	Scienze a livello PhD	93.2%	91.3%	~85%	79.9%	87.6%
MMLU-Pro	Conoscenza generale avanzata	89.6%	91.1%	~88%	~85%	92.0%
HumanEval	Generazione codice	95.0%	95.0%	~92%	~90%	99.0%
Chatbot Arena Elo	Preferenza umana	~1440	~1430	~1410	~1421	1447

(*) DeepSeek V3.2-Speciale, versione ottimizzata. **Nota:** I valori contrassegnati con ~ sono stime basate su fonti multiple e possono variare a seconda della configurazione (reasoning effort, numero di tentativi). I benchmark si saturano rapidamente: AIME 2025 e MMLU-Pro sono ormai quasi al tetto per i modelli migliori.

SEZIONE 3: IL FATTORE COSTO-EFFICACIA

La tabella prezzi da sola è fuorviante senza considerare il costo per task completato. Un modello più costoso per token può costare meno complessivamente se risolve il problema al primo tentativo o con meno token.

Esempio di costo stimato per 1 miliardo di token di output al mese:

Modello	Costo mensile stimato
DeepSeek V3.1	~\$750
Gemini 2.5 Pro	~\$10.000
GPT-5.2	~\$14.000
Claude Sonnet 4.5	~\$15.000
Claude Opus 4.5	~\$25.000
GPT-5.2 Pro	~\$168.000

La forbice è di oltre 200× tra il modello più economico e il più costoso. Questa differenza è il motivo per cui la strategia di routing dei modelli (usare modelli piccoli per task semplici, grandi per quelli complessi) è diventata una pratica standard in produzione.

SEZIONE 4: COME SCEGLIERE – STRATEGIE PRATICHE

- La regola d'oro: partire dal modello più economico che soddisfa i requisiti di qualità, poi salire di livello solo dove necessario. Per coding e software engineering: Claude Opus 4.5/4.6 e GPT-5.2 guidano su SWE-bench. Per progetti con budget limitato, DeepSeek V3.2 e Kimi K2.5 offrono prestazioni sorprendenti.
- Per ragionamento matematico e logico: GPT-5.2 e Kimi K2.5 sono al vertice. DeepSeek R1 offre un rapporto qualità/prezzo eccezionale per task di reasoning.
- Per contesti lunghi (documenti, codebase): Gemini 2.5 Pro con finestra da 1M token è imbattibile. DeepSeek e Kimi offrono 128-262K.
- Per uso generale ad alto volume: DeepSeek V3.1 a \$0.15/\$0.75 rende possibili applicazioni prima economicamente infattibili. Gemini Flash-Lite (\$0.10/\$0.40) è ancora più economico per task semplici.
- Per massima qualità senza vincoli di budget: GPT-5.2 Pro o Claude Opus 4.6, a seconda del dominio.

SEZIONE 5: TENDENZE CHIAVE

- Commoditizzazione della frontiera: La differenza di prestazioni tra i top 10 modelli si è ridotta drasticamente. Scegliere il modello "migliore" conta meno della scelta "giusta per il task".
- Crollo dei prezzi: Il costo per milione di token è sceso dell'80% in 18 mesi. Claude Opus 4.5 costa il 67% in meno del predecessore Opus 4.1.
- Open-weight competitivi: DeepSeek, Kimi K2.5 e Qwen3 competono con i modelli proprietari a una frazione del costo, cambiando l'equazione economica per startup e PMI.
- Model routing come standard: Le architetture di produzione moderne non usano un singolo modello ma un "cascata" di modelli (economico → potente) con routing intelligente basato sulla complessità della richiesta.
- Il contesto lungo è il nuovo differenziatore: Con le prestazioni che convergono, la dimensione della context window (1M token per Gemini) e l'efficienza nel gestirla diventano fattori di scelta sempre più rilevanti.

FONTI DI RIFERIMENTO

- Per dati aggiornati in tempo reale: [pricepertoken.com](#) (prezzi)
- [lmcouncil.ai/benchmarks](#) (benchmark)
- [lmarena.ai](#) (Chatbot Arena Elo)
- [artificialanalysis.ai](#) (velocità e latenza).

Il Costo Ambientale di una Query LLM

Quanto consuma in energia, acqua e CO2 una singola domanda a un modello? E perché il vero problema è la scala.

E **Energia**
~0.3 Wh / query

Equivale a: **9 secondi di TV**, oppure 2 minuti di lampadina LED, oppure 1 secondo di forno

3-10x una ricerca Google tradizionale. Su scala: solo ChatGPT consuma come **35.000 abitazioni/anno**

A **Acqua**
~5 mL / query (totale)

Equivale a: **1 cucchiaino da caffè** (0.3 mL raffreddamento diretto + ~5 mL includendo l'acqua per generare elettricità)

28.000x meno di una tazza di caffè. Su scala: consumo annuo ChatGPT = fabbisogno di **1.2 milioni di persone**

C **CO2**
~0.1 g / query

Equivale a: **mezzo metro in auto**, oppure 1-4 respiri umani, oppure mezza email

Stesso ordine di grandezza di una ricerca Google. Su scala: **decine di migliaia di tonnellate/anno**

RIEPILOGO: PER SINGOLA QUERY VS SCALA GLOBALE

RISORSA	PER QUERY	EQUIVALENTE QUOTIDIANO	PROBLEMA A SCALA GLOBALE
Energia	~0.3 Wh	9 secondi di TV	35.000 case/anno (solo ChatGPT)
Acqua	~5 mL	1 cucchiaino di caffè	Fabbisogno di 1.2M persone/anno
CO2	~0.1 g	Mezzo metro in auto	Decine di migliaia di t/anno

NON TUTTE LE QUERY SONO UGUALI



IL PARADOSSO DI JEVONS



L'efficienza per singola query migliora costantemente: Gemini ha ridotto il consumo energetico per query di **44x in un solo anno**.
Ma il consumo totale dei provider continua a **crescere**, perché la domanda esplode più velocemente dei guadagni di efficienza.
E il paradosso di Jevons: **rendere qualcosa più efficiente ne aumenta l'uso totale**, perché abbassa la barriera d'accesso.

IL PUNTO CHIAVE

Il singolo uso è irrilevante. L'impatto nasce dalla scala.

Servirebbero **~1.000 query al giorno** per avere un impatto percepibile sul consumo personale. Ma con **oltre un miliardo di query/giorno** a livello globale, l'impatto aggregato è significativo e in crescita rapida.

Cervello Umano vs LLM

Un confronto per mettere in prospettiva: cosa sappiamo fare noi e cosa sanno fare loro

	Cervello Umano ~1.4 kg, evoluzione di milioni di anni	LLM (es. GPT-4) Data center, addestramento di mesi
Conessioni / Parametri	~100.000 miliardi di sinapsi Ogni neurone connesso a migliaia di altri, formando reti dinamiche che si riorganizzano continuamente	~1.700 miliardi di parametri Pesi statici fissati dopo il training. Nessuna riorganizzazione autonoma dopo il deployment Cervello: ~60x più connessioni
Memoria	10 – 2.500 Terabyte Memoria a lungo termine persistente, episodica, semantica. Ricordi che durano una vita intera	Solo la finestra di contesto Nessuna memoria persistente autonoma. Tutto ciò che sfiora il contesto viene dimenticato Cervello: memoria permanente
Dati per apprendere	Pochi milioni di parole Un bambino impara a parlare con l'esperienza diretta del mondo: vista, tatto, suoni, emozioni	Migliaia di miliardi di token Solo testo (e immagini). Nessuna esperienza sensoriale, nessuna interazione con il mondo fisico Cervello: ~1.000.000x più efficiente
Comprensione del mondo	Esperienza incarnata "Sa" che il fuoco scotta perché prova dolore. Conoscenza radicata nel corpo e nell'esperienza	Correlazioni statistiche Simula comprensione tramite pattern linguistici. Non "sa" cosa significa caldo — sa che appare vicino a fuoco Cervello: comprensione vs simulazione
Consumo energetico	~20 Watt Come una lampadina LED. 24 ore su 24, per tutta la vita. Esegue l'equivalente di un exaflop con 20W 1.000.000x più efficiente	~0.3 Wh per query (inferenza) Training di GPT-4: ~50 GWh (3 giorni di energia di San Francisco). I data center AI consumano Gigawatt

Il dato che mette tutto in prospettiva

Il cervello esegue l'equivalente di **un exaflop** (10^{18} operazioni/secondo) con soli 20 Watt. Il supercomputer Frontier raggiunge lo stesso throughput con **20 Megawatt** — un milione di volte più energia. Nonostante tutto il progresso dell'AI, la biologia resta incomparabilmente più efficiente.

20 W vs 20 MW

Cervello vs Supercomputer per 1 exaflop

Riferimenti

Esecuzione locale di LLM:

- [llama.cpp](#) — il motore open-source per eseguire LLM in locale
- [Ollama](#) — CLI per scaricare e lanciare modelli GGUF con un comando
- [LM Studio](#) — interfaccia grafica per LLM locali, supporta GGUF e tutti i modelli open-weight
- [Hugging Face](#) — il repository principale per modelli, dataset e GGUF pre-quantizzati

Quantizzazione e formati:

- [Specifica GGUF](#) — documentazione ufficiale del formato
- [bartowski su HF](#) — uno dei principali curatori di modelli GGUF pre-quantizzati

Benchmark e confronto modelli:

- [LM Arena](#) — Chatbot Arena con classifica Elo basata su preferenze umane
- [Artificial Analysis](#) — confronto velocità, latenza e prezzi API in tempo reale
- [LM Council](#) — benchmark indipendenti aggregati
- [Price Per Token](#) — comparatore prezzi API aggiornato

Costo ambientale dell'AI:

- [Google \(Ago 2025\). "Measuring the environmental impact of delivering AI at Google Scale"](#) — prima disclosure dettagliata per-query
- [Altman, S. \(Giu 2025\). "The Gentle Singularity"](#) — prima dichiarazione OpenAI su energia e acqua per query
- [Jegham, N. et al. \(2025\). "How Hungry is AI?"](#) — benchmark indipendente energia/acqua/CO2 su 30 modelli (arXiv)
- Li, P. et al. (2025). "Making AI Less Thirsty" — Communications of the ACM, 68(7)

Distillazione e compressione:

- [Hinton, G. et al. \(2015\). "Distilling the Knowledge in a Neural Network"](#) — il paper fondativo sulla knowledge distillation
- [DeepSeek-R1 Technical Report \(2025\)](#) — esempio recente di distillazione su larga scala con reasoning traces

Dimensionamento hardware:

- [Can I Run This LLM?](#) — calcolatore VRAM di Hugging Face
- [NVIDIA GeForce Specs](#) — specifiche ufficiali di tutte le GPU consumer